ARCHITECTURE TECHNIQUE

MÉDIDOC

Adaptation du cas « Clinique privée »

Architecture • API • sécurité • déploiement • évolution

BESOINS ARCHITECTURE INFRASTRUCTURE SERVICES

FIL ROUGE : partir du besoin avant la technologie

Pourquoi adapter le sujet clinique à Médidoc ?

Le sujet « Système d'information d'une clinique privée » sert ici de **terrain d'approfondissement**. Il ne remplace pas le sujet officiel attribué au groupe. L'objectif est de tester Médidoc face à des contraintes concrètes : données sensibles, rôles multiples, accès distant, sauvegarde, traçabilité et exploitation par une petite équipe IT.

La logique du support principal est conservée : BESOINS → ARCHITECTURE → INFRASTRUCTURE → SERVICES.

Ce que le cas clinique renforce dans Médidoc

DONNÉES SENSIBLES	RBAC, moindre privilège, sessions sécurisées et audit deviennent structurants.
PETITE ÉQUIPE IT	La simplicité d'exploitation favorise un monolithe modulaire plutôt qu'une distribution prématurée.
ACCÈS DISTANT	Les accès externes passent par une frontière sécurisée ; le backend contrôle les droits.
CONTINUITÉ	Backup automatisé + restauration testée + objectifs RPO/RTO à définir.

Décision de travail

Architecture 3-tiers, organisée en couches, déployée initialement comme monolithe modulaire avec API REST. PostgreSQL conserve les données structurées et un stockage séparé les documents sources.

1. Comprendre le système avant de le construire

La valeur centrale de Médidoc est de permettre à un professionnel autorisé de retrouver rapidement un patient, comprendre son parcours chronologique, consulter les documents sources et enregistrer les événements nécessaires, tout en respectant les droits d'accès et la traçabilité.

Exigences fonctionnelles adaptées

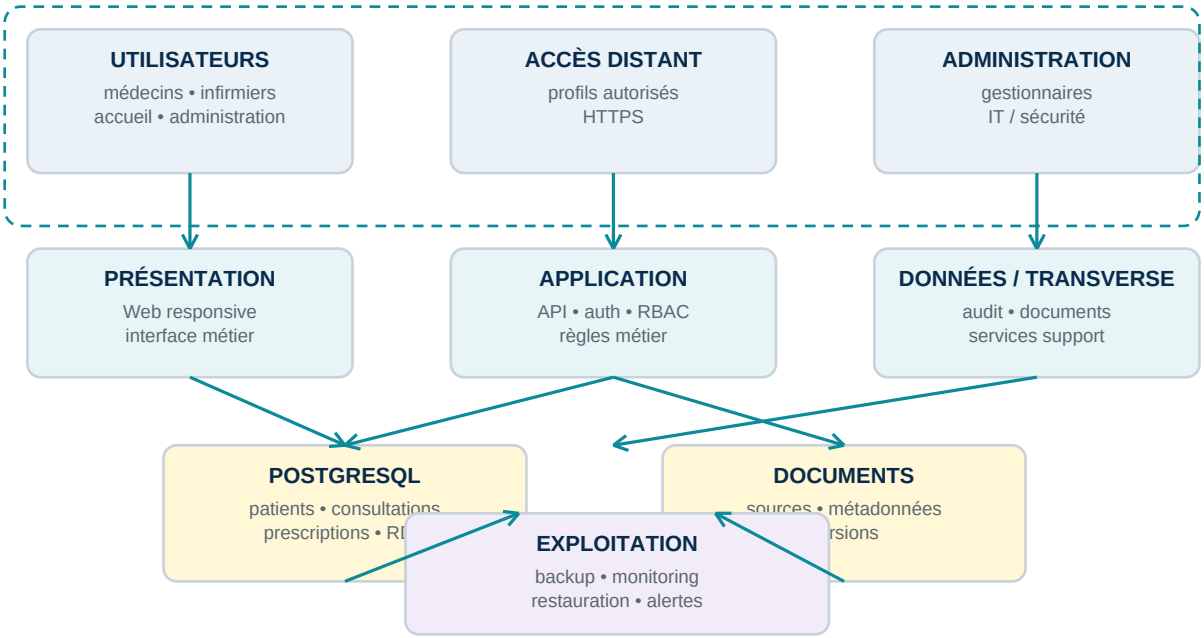
PATIENTS	Identité, recherche, détection des doublons et fusion contrôlée.
DOSSIER PATIENT	Synthèse, antécédents, allergies, problèmes, timeline et documents.
CONSULTATIONS	Motif, histoire, constantes, observations, examen clinique et compte rendu.
PRESCRIPTIONS	Médicament, posologie, fréquence, durée, instructions et statut.
RENDEZ-VOUS / EXAMENS	Agenda, demandes, résultats et pièces jointes selon le périmètre validé.
AUDIT	Traçabilité des accès et des actions critiques.

Contraintes non fonctionnelles

SÉCURITÉ	TLS, authentification, RBAC, moindre privilège et protection des sessions.
CONFIDENTIALITÉ	Protection des données au repos selon l'hébergement ; secrets hors Git.
RÉSILIENCE	Backups automatisés, chiffrés et testés ; RPO/RTO à définir.
EXPLOITATION	Logs structurés, métriques, alertes et environnements séparés.
INTEROPÉRABILITÉ	API versionnées, OpenAPI et étude progressive de FHIR/DICOM.

2. Pourquoi 3-tiers ?

Le choix 3-tiers sépare la présentation, le traitement applicatif et les données. Pour Médidoc, les postes utilisateurs ne doivent ni porter les règles métier ni accéder directement au SGBD.



Lecture : l'API/backend devient la frontière de confiance : elle authentifie, autorise, applique les règles métier et journalise les actions sensibles. La base et le stockage documentaire restent derrière cette couche.

Pourquoi pas 2-tiers ?

Le 2-tiers peut être simple dans un petit environnement local, mais il rapproche les clients des données et complique la centralisation de la sécurité, des règles métier et de l'accès distant. Le 3-tiers apporte cette séparation sans imposer la complexité des microservices.

3. Monolithe ne signifie pas désordre

Le choix porte sur un **monolithe modulaire** : les domaines sont séparés conceptuellement et dans le code, mais déployés ensemble au départ.

Patients	Identité, recherche, doublons	Utilisateurs / RBAC
Consultations	Événements cliniques, observations	Patients + utilisateurs
Prescriptions	Prescriptions et statuts	Patients + consultations
Rendez-vous	Agenda et suivi	Patients + utilisateurs
Documents	Documents, métadonnées, versions	Patients + consultations + RBAC
Audit	Accès et actions critiques	Modules sensibles
Utilisateurs / RBAC	Identités, rôles, permissions	—

Règle d’évolution

Un microservice n’est extrait que lorsqu’une contrainte mesurée le justifie : charge spécifique, déploiement autonome, isolation, responsabilité distincte ou besoin d’évolution indépendant.

4. Les endpoints deviennent des capacités métier

Pour Médidoc, les endpoints sont alignés sur les parcours réels du dossier patient. Une API ne doit pas être confondue avec la base de données : elle expose une capacité contrôlée par le backend.

POST	/api/v1/auth/login	Authentifier un utilisateur
GET	/api/v1/patients/{id}	Consulter un patient autorisé
POST	/api/v1/patients	Créer un patient
GET	/api/v1/patients/{id}/timeline	Consulter la timeline
POST	/api/v1/consultations	Créer une consultation
PATCH	/api/v1/consultations/{id}	Modifier une consultation
POST	/api/v1/prescriptions	Créer une prescription
GET	/api/v1/exams/{id}/results	Consulter un résultat
POST	/api/v1/documents	Associer un document source
GET	/api/v1/audit-events	Consulter l’audit

Exemple : GET /api/v1/patients/125 → identité, rôle et contexte d’accès sont vérifiés avant toute lecture.

5. La sécurité doit être visible dans l’architecture

Dans une clinique, la sécurité influence le réseau, l’authentification, les APIs, les données, les journaux et les sauvegardes.

AUTHENTIFICATION	Identifier l’utilisateur et protéger les sessions.
AUTORISATION	RBAC + moindre privilège ; contrôle côté backend.
CHIFFREMENT	HTTPS/TLS ; protection au repos selon l’hébergement.
AUDIT	Accès et modifications sensibles avec auteur et horodatage.
SAUVEGARDE	Automatiser, chiffrer, isoler et tester les restaurations.
SUPERVISION	Logs, métriques, alertes, disponibilité et capacité.
SECRETS	Ne jamais stocker mots de passe, clés ou tokens dans Git.

Principe d’exploitation

Une architecture professionnelle doit être observable et récupérable : diagnostiquer, restaurer et revenir à une version stable.

6. Vérifier l’architecture par un scénario réel

Scénario : un médecin ouvre le dossier d’un patient, ajoute une observation et une prescription ; la modification doit être enregistrée et tracée.

01	Médecin → interface	Demande d’ouverture du dossier
02	Interface → API	GET /api/v1/patients/{id}
03	API → sécurité	Identité + rôle + contexte
04	API → données	Lecture via couche d’accès
05	API → interface	Retour des informations autorisées
06	Médecin → API	Création/modification consultation
07	Backend	Validation des règles métier
08	Backend → PostgreSQL	Enregistrement événement clinique
09	Backend → audit	Journalisation action critique
10	Backend → interface	Confirmation

Ce scénario démontre : utilisateur → présentation → API → sécurité → logique métier → données → audit → retour utilisateur.

7. Construire aujourd’hui sans bloquer demain

Le cas clinique pousse à penser à la croissance, mais la réponse n’est pas automatiquement « microservices ». L’évolution doit être progressive et guidée par les mesures.

1 — MVP	Monolithe modulaire + API REST + PostgreSQL + stockage documentaire + RBAC + audit + monitoring + CI/CD.
2 — MESURER	Temps de réponse, volumes, erreurs, capacité, disponibilité et besoins réels.
3 — OPTIMISER	Indexation, requêtes, cache, réplication ou répartition de charge si nécessaire.
4 — EXTRAIRE	Service indépendant seulement si une contrainte mesurée le justifie.
5 — IA	Séparer les modèles IA du cœur clinique ; validation humaine et gouvernance adaptées.

Questions d’architecte

- Où est le goulot d’étranglement ?
- Quelle contrainte impose l’évolution ?
- Peut-on résoudre le problème sans distribuer le système ?
- Quel nouveau risque introduit le choix ?
- Comment mesurer que la nouvelle architecture est meilleure ?

Conclusion

La maturité architecturale ne se mesure pas au nombre de composants. Elle se mesure à la capacité de relier chaque décision à un besoin, une contrainte, un risque et une conséquence.

Références : support principal « Architecture Technique des Systèmes Informatiques », sujet clinique et cahiers des charges Médidoc V2.0/V2.1. Les choix propres à Médidoc restent à valider avec la structure pilote.